

FACULDADE COTEMIG

FUNDAMENTOS TEÓRICOS DA COMPUTAÇÃO

**RELATÓRIO TÉCNICO: SIMULADOR DE MÁQUINAS
ABSTRATAS**

Membros da Equipe:

Alex Paulo Vianna Da Costa Candido e Silva

Carlos Henrique Queiroz Teixeira Ramos

Bernardo Medeiros Mendes

Belo Horizonte / MG

Junho de 2026

1 Introdução

O presente relatório documenta o desenvolvimento de um simulador computacional de máquinas abstratas, requisito para a avaliação final da disciplina de Fundamentos Teóricos da Computação. O objetivo central deste trabalho é transpor modelos matemáticos teóricos — Autômatos Finitos Determinísticos (AFD), Autômatos de Pilha (AP) e Máquinas de Turing (MT) — para algoritmos executáveis em linguagem C#.

O estudo de linguagens formais e autômatos fornece a base matemática para a compreensão do que pode ou não ser computado. Através deste projeto, buscou-se validar conceitos como determinismo, manipulação de memória restrita (pilhas) e memória irrestrita (fita infinita). O sistema foi arquitetado de forma modular, com validações baseadas nas tuplas formais que definem cada máquina, permitindo não apenas o reconhecimento de linguagens específicas, mas também a resolução de problemas de decisão e funções matemáticas (como a adição unária).

2 Fundamentação Teórica

2.1 Autômato Finito Determinístico (AFD)

Um AFD é um modelo matemático restrito que reconhece Linguagens Regulares, formalmente definido por uma 5-tupla $M = (Q, \Sigma, \delta, q_0, F)$, onde: Q é um conjunto finito de estados; Σ é o alfabeto de entrada; $\delta : Q \times \Sigma \rightarrow Q$ é a função de transição; $q_0 \in Q$ é o estado inicial; e $F \subseteq Q$ é o conjunto de estados finais. **Exemplo de Cálculo (L_1 - Termina em ab):** Ao processar a palavra $w = aab$, a máquina parte de q_0 . Lê ' a ' $\rightarrow q_1$. Lê ' a ' $\rightarrow q_1$. Lê ' b ' $\rightarrow q_2$. Como $q_2 \in F$, a palavra é aceita.

2.2 Autômato de Pilha (AP)

Um AP reconhece Linguagens Livres de Contexto, possuindo uma memória auxiliar LIFO. É definido pela 7-tupla $M = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$. No nosso projeto, a aceitação ocorre *exclusivamente por pilha vazia* (logo, $F = \emptyset$). **Exemplo de Cálculo ($L_2 = a^n b^n$):** Para $w = aabb$, a máquina lê ' a ' e empilha ' A ' duas vezes. Através de um λ -movimento, adivinha a transição para os ' b 's e, a cada ' b ' lido, desempilha um ' A '. Ao final da cadeia, a pilha contém apenas Z_0 , que é desempilhado, aceitando a palavra.

2.3 Máquina de Turing (MT)

A MT é o modelo mais poderoso, capaz de reconhecer Linguagens Recursivamente Enumeráveis, possuindo uma fita infinita. Definida por $M = (Q, \Sigma, \Gamma, \delta, q_0, B, F)$. **Exemplo de Cálculo ($L_4 = a^n b^n c^n$):** Ao processar abc , a MT lê ' a ', escreve ' X ' e move para a direita (R) até achar ' b ', escreve ' Y ' e move (R) até achar ' c ', escrevendo ' Z '. Em seguida, move para a esquerda

(L) para reiniciar a marcação. Se encontrar apenas brancos (B) no final do processo, aceita a palavra.

3 Descrição da Implementação

O projeto foi desenvolvido em .NET (C#), priorizando o mapeamento direto da teoria para estruturas de dados modernas.

3.1 Implementação do AFD

O AFD foi estruturado utilizando a classe `HashSet<T>` do C# para representar os conjuntos finitos matemáticos (Q, Σ, F) , garantindo unicidade dos elementos e complexidade de busca $O(1)$. A função de transição δ foi modelada como um `Dictionary<(string, char), string>`, mapeando perfeitamente a relação estado-símbolo para um novo estado. Além das validações no código (linguagem L_1), foi implementado o Desafio Obrigatório que carrega um AFD genérico via arquivo JSON (`System.Text.Json`).

Questão Reflexiva (Não-totalidade e Entradas Inválidas): No nosso simulador, a não-totalidade da função δ foi tratada simulando um “estado de poço” (*trap state*). Se a máquina está num estado e lê um símbolo não mapeado, o método `TryGetValue` falha, rejeitando a cadeia. Para símbolos fora do alfabeto (Σ), implementamos uma trava estrutural prévia de verificação; caso o caractere não pertença a Σ , o processamento é abortado imediatamente.

3.2 Implementação do Autômato de Pilha

Para o AP, a fita auxiliar foi implementada estritamente com a classe nativa `Stack<char>`. Para suportar o não-determinismo exigido pelo desafio dos Palíndromos (L_3), a função de transição precisou retornar uma `List` de caminhos possíveis, associada a uma simulação recursiva de estados.

Questão Reflexiva (Complexidade L_2 vs L_3): A diferença de complexidade reside na necessidade do Não-Determinismo. A linguagem $L_2 = \{a^n b^n\}$ é determinística (reconhecida por um APD): o primeiro ‘b’ engatilha o desempilhamento. Já L_3 (palíndromos) exige um Autômato de Pilha Não-Determinístico (APND), pois a máquina precisa de λ -movimentos para “adivinhar” o meio da palavra. Como nos APs o não-determinismo é estritamente mais poderoso que o determinismo, L_3 possui uma complexidade superior a L_2 .

3.3 Implementação da Máquina de Turing

A maior inovação do projeto ocorreu na implementação da Fita Infinita da Máquina de Turing. Optou-se por utilizar um `Dictionary<int, char>`, onde a chave é o índice da fita (que pode

assumir valores negativos infinitos) e o valor é o símbolo. Isso evita os problemas de redimensionamento de memória que *Arrays* ou *Lists* apresentariam.

Questão Reflexiva (Adição Unária vs Tese de Church-Turing): A nossa Máquina de Turing que calcula $f(n) = n + 1$ é um algoritmo formal. A Tese de Church-Turing postula que qualquer processo computacional mecânico descrito por um conjunto finito de regras pode ser simulado por uma MT. A nossa implementação possui instruções não-ambíguas, processa os dados passo a passo de forma determinística e, crucialmente, atinge a condição de parada (*halt state*), constituindo a materialização de um algoritmo.

4 Resultados e Testes

As saídas no terminal (Console) corroboram a corretude matemática das máquinas construídas. Os rastreamentos instantâneos das memórias (traces) mostraram-se exatos. Abaixo, os resultados processados na íntegra para os arquivos de entrada exigidos:

AFD (Parte 1)

Diagrama de Transições (Tabela)

Estado	a	b
q0	q1	q0
q1	q1	q2
q2	q1	q0

Processando entradas.txt para L1...

Cadeia: ab

Rastro: q0 -> q1 -> q2

Resultado: ACEITA

Cadeia: aab

Rastro: q0 -> q1 -> q1 -> q2

Resultado: ACEITA

Cadeia: bab

Rastro: q0 -> q0 -> q1 -> q2

Resultado: ACEITA

Cadeia: ababab

Rastro: q0 -> q1 -> q2 -> q1 -> q2 -> q1 -> q2

Resultado: ACEITA

Cadeia: ba
 Rastro: q0 -> q0 -> q1
 Resultado: REJEITA

 Cadeia: vazia
 Rastro: q0
 Resultado: REJEITA

 Cadeia: b
 Rastro: q0 -> q0
 Resultado: REJEITA

 Carregando afd.json
 Máquina carregada com sucesso do arquivo JSON!
 Diagrama de Transições (Tabela)

Estado	0	1
q0	q0	q1
q1	q1	q0

=== Simulador de Autômato de Pilha (Parte 2) ===

=== L2: $a^n b^n$ ($n \geq 1$) ===

Processando: ab
 Configuração: (q: q0, w: ab, Pilha: Z)
 Configuração: (q: q0, w: b, Pilha: AZ)
 Configuração: (q: q1, w: b, Pilha: AZ)
 Configuração: (q: q1, w: E, Pilha: Z)
 Configuração: (q: q2, w: E, Pilha: [Vazia])
 Resultado: ACEITA

 Processando: aabb
 Configuração: (q: q0, w: aabb, Pilha: Z)
 Configuração: (q: q0, w: abb, Pilha: AZ)
 Configuração: (q: q1, w: abb, Pilha: AZ)
 Configuração: (q: q0, w: bb, Pilha: AAZ)
 Configuração: (q: q1, w: bb, Pilha: AAZ)
 Configuração: (q: q1, w: b, Pilha: AZ)
 Configuração: (q: q1, w: E, Pilha: Z)
 Configuração: (q: q2, w: E, Pilha: [Vazia])
 Resultado: ACEITA

Processando: aaabbb

Configuração: (q: q0, w: aaabbb, Pilha: Z)
Configuração: (q: q0, w: aabbb, Pilha: AZ)
Configuração: (q: q1, w: aabbb, Pilha: AZ)
Configuração: (q: q0, w: abbb, Pilha: AAZ)
Configuração: (q: q1, w: abbb, Pilha: AAZ)
Configuração: (q: q0, w: bbb, Pilha: AAAZ)
Configuração: (q: q1, w: bbb, Pilha: AAAZ)
Configuração: (q: q1, w: bb, Pilha: AAZ)
Configuração: (q: q1, w: b, Pilha: AZ)
Configuração: (q: q1, w: E, Pilha: Z)
Configuração: (q: q2, w: E, Pilha: [Vazia])

Resultado: ACEITA

Processando: aab

Configuração: (q: q0, w: aab, Pilha: Z)
Configuração: (q: q0, w: ab, Pilha: AZ)
Configuração: (q: q1, w: ab, Pilha: AZ)
Configuração: (q: q0, w: b, Pilha: AAZ)
Configuração: (q: q1, w: b, Pilha: AAZ)
Configuração: (q: q1, w: E, Pilha: AZ)

Resultado: REJEITA

Processando: abb

Configuração: (q: q0, w: abb, Pilha: Z)
Configuração: (q: q0, w: bb, Pilha: AZ)
Configuração: (q: q1, w: bb, Pilha: AZ)
Configuração: (q: q1, w: b, Pilha: Z)
Configuração: (q: q2, w: b, Pilha: [Vazia])

Resultado: REJEITA

Processando: ba

Configuração: (q: q0, w: ba, Pilha: Z)

Resultado: REJEITA

Processando: vazia

Configuração: (q: q0, w: E, Pilha: Z)

Resultado: REJEITA

Processando: abab

Configuração: (q: q0, w: abab, Pilha: Z)

Configuração: (q: q0, w: bab, Pilha: AZ)
 Configuração: (q: q1, w: bab, Pilha: AZ)
 Configuração: (q: q1, w: ab, Pilha: Z)
 Configuração: (q: q2, w: ab, Pilha: [Vazia])
 Resultado: REJEITA

=== Desafio L3: Palíndromos sobre {a,b} ===

Processando: a

Configuração: (q: q0, w: a, Pilha: Z)
 Configuração: (q: q1, w: a, Pilha: Z)
 Configuração: (q: q2, w: a, Pilha: [Vazia])
 Configuração: (q: q0, w: E, Pilha: AZ)
 Configuração: (q: q1, w: E, Pilha: AZ)
 Configuração: (q: q1, w: E, Pilha: Z)
 Configuração: (q: q2, w: E, Pilha: [Vazia])
 Resultado: ACEITA

Processando: aba

Configuração: (q: q0, w: aba, Pilha: Z)
 Configuração: (q: q1, w: aba, Pilha: Z)
 Configuração: (q: q2, w: aba, Pilha: [Vazia])
 Configuração: (q: q0, w: ba, Pilha: AZ)
 Configuração: (q: q1, w: ba, Pilha: AZ)
 Configuração: (q: q0, w: a, Pilha: BAZ)
 Configuração: (q: q1, w: a, Pilha: BAZ)
 Configuração: (q: q0, w: E, Pilha: ABAZ)
 Configuração: (q: q1, w: E, Pilha: ABAZ)
 Configuração: (q: q1, w: E, Pilha: BAZ)
 Configuração: (q: q1, w: a, Pilha: AZ)
 Configuração: (q: q1, w: E, Pilha: Z)
 Configuração: (q: q2, w: E, Pilha: [Vazia])
 Configuração: (q: q1, w: ba, Pilha: Z)
 Configuração: (q: q2, w: ba, Pilha: [Vazia])
 Resultado: ACEITA

Processando: abba

Configuração: (q: q0, w: abba, Pilha: Z)
 Configuração: (q: q1, w: abba, Pilha: Z)
 Configuração: (q: q2, w: abba, Pilha: [Vazia])
 Configuração: (q: q0, w: bba, Pilha: AZ)
 Configuração: (q: q1, w: bba, Pilha: AZ)
 Configuração: (q: q0, w: ba, Pilha: BAZ)
 Configuração: (q: q1, w: ba, Pilha: BAZ)

Configuração: (q: q1, w: a, Pilha: AZ)
 Configuração: (q: q1, w: E, Pilha: Z)
 Configuração: (q: q2, w: E, Pilha: [Vazia])
 Configuração: (q: q1, w: ba, Pilha: AZ)
 Configuração: (q: q1, w: bba, Pilha: Z)
 Configuração: (q: q2, w: bba, Pilha: [Vazia])
 Resultado: ACEITA

Processando: ab

Configuração: (q: q0, w: ab, Pilha: Z)
 Configuração: (q: q1, w: ab, Pilha: Z)
 Configuração: (q: q2, w: ab, Pilha: [Vazia])
 Configuração: (q: q0, w: b, Pilha: AZ)
 Configuração: (q: q1, w: b, Pilha: AZ)
 Configuração: (q: q0, w: E, Pilha: BAZ)
 Configuração: (q: q1, w: E, Pilha: BAZ)
 Configuração: (q: q1, w: E, Pilha: AZ)
 Configuração: (q: q1, w: b, Pilha: Z)
 Configuração: (q: q2, w: b, Pilha: [Vazia])
 Resultado: REJEITA

Processando: aab

Configuração: (q: q0, w: aab, Pilha: Z)
 Configuração: (q: q1, w: aab, Pilha: Z)
 Configuração: (q: q2, w: aab, Pilha: [Vazia])
 Configuração: (q: q0, w: ab, Pilha: AZ)
 Configuração: (q: q1, w: ab, Pilha: AZ)
 Configuração: (q: q1, w: b, Pilha: Z)
 Configuração: (q: q2, w: b, Pilha: [Vazia])
 Configuração: (q: q0, w: b, Pilha: AAZ)
 Configuração: (q: q1, w: b, Pilha: AAZ)
 Configuração: (q: q0, w: E, Pilha: BAAZ)
 Configuração: (q: q1, w: E, Pilha: BAAZ)
 Configuração: (q: q1, w: E, Pilha: AAZ)
 Configuração: (q: q1, w: b, Pilha: AZ)
 Configuração: (q: q1, w: ab, Pilha: Z)
 Configuração: (q: q2, w: ab, Pilha: [Vazia])
 Resultado: REJEITA

=== Simulador de Máquina de Turing (Parte 3) ===

=== L4: $a^n b^n c^n$ ($n \geq 1$) ===

Processando L4: abc


```

Fita: ... _ [q0>a] b c ...
Fita: ... X [q1>b] c ...
Fita: ... X Y [q2>c] _ ...
Fita: ... X [q3>Y] Z ...
Fita: ... _ [q3>X] Y Z ...
Fita: ... X [q0>Y] Z ...
Fita: ... X Y [q4>Z] _ ...
Fita: ... X Y Z [q4>_] _ ...
Fita: ... X Y Z _ [q_acc>_] _ ...
Resultado: ACEITA

```

Processando L4: aabbcc

```

Fita: ... _ [q0>a] a b b c c ...
Fita: ... X [q1>a] b b c c ...
Fita: ... X a [q1>b] b c c ...
Fita: ... X a Y [q2>b] c c ...
Fita: ... X a Y b [q2>c] c ...
Fita: ... X a Y [q3>b] Z c ...
Fita: ... X a [q3>Y] b Z c ...
Fita: ... X [q3>a] Y b Z c ...
Fita: ... _ [q3>X] a Y b Z c ...
Fita: ... X [q0>a] Y b Z c ...
Fita: ... X X [q1>Y] b Z c ...
Fita: ... X X Y [q1>b] Z c ...
Fita: ... X X Y Y [q2>Z] c ...
Fita: ... X X Y Y Z [q2>c] _ ...
Fita: ... X X Y Y [q3>Z] Z ...
Fita: ... X X Y [q3>Y] Z Z ...
Fita: ... X X [q3>Y] Y Z Z ...
Fita: ... X [q3>X] Y Y Z Z ...
Fita: ... X X [q0>Y] Y Z Z ...
Fita: ... X X Y [q4>Y] Z Z ...
Fita: ... X X Y Y [q4>Z] Z ...
Fita: ... X X Y Y Z [q4>Z] _ ...
Fita: ... X X Y Y Z Z [q4>_] _ ...
Fita: ... X X Y Y Z Z _ [q_acc>_] _ ...
Resultado: ACEITA

```

Processando L4: aaabbbccc

```

Fita: ... _ [q0>a] a a b b b c c c ...
Fita: ... X [q1>a] a b b b c c c ...
Fita: ... X a [q1>a] b b b c c c ...
Fita: ... X a a [q1>b] b b c c c ...
Fita: ... X a a Y [q2>b] b c c c ...
Fita: ... X a a Y b [q2>b] c c c ...

```

```

Fita: ... X a a Y b b [q2>c] c c ...
Fita: ... X a a Y b [q3>b] Z c c ...
Fita: ... X a a Y [q3>b] b Z c c ...
Fita: ... X a a [q3>Y] b b Z c c ...
Fita: ... X a [q3>a] Y b b Z c c ...
Fita: ... X [q3>a] a Y b b Z c c ...
Fita: ... _ [q3>X] a a Y b b Z c c ...
Fita: ... X [q0>a] a Y b b Z c c ...
Fita: ... X X [q1>a] Y b b Z c c ...
Fita: ... X X a [q1>Y] b b Z c c ...
Fita: ... X X a Y [q1>b] b Z c c ...
Fita: ... X X a Y Y [q2>b] Z c c ...
Fita: ... X X a Y Y b [q2>Z] c c ...
Fita: ... X X a Y Y b Z [q2>c] c ...
Fita: ... X X a Y Y b [q3>Z] Z c ...
Fita: ... X X a Y Y [q3>b] Z Z c ...
Fita: ... X X a Y [q3>Y] b Z Z c ...
Fita: ... X X a [q3>Y] Y b Z Z c ...
Fita: ... X X [q3>a] Y Y b Z Z c ...
Fita: ... X [q3>X] a Y Y b Z Z c ...
Fita: ... X X [q0>a] Y Y b Z Z c ...
Fita: ... X X X [q1>Y] Y b Z Z c ...
Fita: ... X X X Y [q1>Y] b Z Z c ...
Fita: ... X X X Y Y [q1>b] Z Z c ...
Fita: ... X X X Y Y Y [q2>Z] Z c ...
Fita: ... X X X Y Y Y Z [q2>Z] c ...
Fita: ... X X X Y Y Y Z Z [q2>c] _ ...
Fita: ... X X X Y Y Y Z [q3>Z] Z ...
Fita: ... X X X Y Y Y [q3>Z] Z Z ...
Fita: ... X X X Y Y [q3>Y] Z Z Z ...
Fita: ... X X X Y [q3>Y] Y Z Z Z ...
Fita: ... X X X [q3>Y] Y Y Z Z Z ...
Fita: ... X X [q3>X] Y Y Y Z Z Z ...
Fita: ... X X X [q0>Y] Y Y Z Z Z ...
Fita: ... X X X Y [q4>Y] Y Z Z Z ...
Fita: ... X X X Y Y [q4>Y] Z Z Z ...
Fita: ... X X X Y Y Y [q4>Z] Z Z ...
Fita: ... X X X Y Y Y Z [q4>Z] Z ...
Fita: ... X X X Y Y Y Z Z [q4>Z] _ ...
Fita: ... X X X Y Y Y Z Z Z [q4>_] _ ...
Fita: ... X X X Y Y Y Z Z Z _ [q_acc>_] _ ...

```

Resultado: ACEITA

Processando L4: aabbc

```

Fita: ... _ [q0>a] a b b c ...
Fita: ... X [q1>a] b b c ...

```

```

Fita: ... X a [q1>b] b c ...
Fita: ... X a Y [q2>b] c ...
Fita: ... X a Y b [q2>c] _ ...
Fita: ... X a Y [q3>b] Z ...
Fita: ... X a [q3>Y] b Z ...
Fita: ... X [q3>a] Y b Z ...
Fita: ... _ [q3>X] a Y b Z ...
Fita: ... X [q0>a] Y b Z ...
Fita: ... X X [q1>Y] b Z ...
Fita: ... X X Y [q1>b] Z ...
Fita: ... X X Y Y [q2>Z] _ ...
Fita: ... X X Y Y Z [q2>_] _ ...
Resultado: REJEITA
-----

```

Processando L4: ab

```

Fita: ... _ [q0>a] b ...
Fita: ... X [q1>b] _ ...
Fita: ... X Y [q2>_] _ ...
Resultado: REJEITA
-----

```

Processando L4: abc abc

```

Fita: ... _ [q0>a] b c a b c ...
Fita: ... X [q1>b] c a b c ...
Fita: ... X Y [q2>c] a b c ...
Fita: ... X [q3>Y] Z a b c ...
Fita: ... _ [q3>X] Y Z a b c ...
Fita: ... X [q0>Y] Z a b c ...
Fita: ... X Y [q4>Z] a b c ...
Fita: ... X Y Z [q4> ] a b c ...
Resultado: REJEITA
-----

```

Processando L4: vazia

```

Fita: ... _ [q0>_] _ ...
Resultado: REJEITA
-----

```

=== Desafio: Computando $f(n) = n + 1$ (Adição Unária) ===

Processando $f(n)$ para $n = 1$ (Cadeia: 1)

```

Fita: ... _ [q0>1] _ ...
Fita: ... 1 [q0>_] _ ...
Fita: ... 1 1 [q_acc>_] _ ...
Resultado computado com sucesso!
-----

```

```

Processando f(n) para n = 2 (Cadeia: 11)
Fita: ... _ [q0>1] 1 ...
Fita: ... 1 [q0>1] _ ...
Fita: ... 1 1 [q0>_] _ ...
Fita: ... 1 1 1 [q_acc>_] _ ...
Resultado computado com sucesso!
-----

Processando f(n) para n = 4 (Cadeia: 1111)
Fita: ... _ [q0>1] 1 1 1 ...
Fita: ... 1 [q0>1] 1 1 ...
Fita: ... 1 1 [q0>1] 1 ...
Fita: ... 1 1 1 [q0>_] _ ...
Fita: ... 1 1 1 1 [q0>_] _ ...
Fita: ... 1 1 1 1 1 [q_acc>_] _ ...
Resultado computado com sucesso!
-----

```

5 Análise Comparativa de Poder Computacional

O desenvolvimento consecutivo destas três máquinas permitiu uma análise clara e prática da Hierarquia de Chomsky.

Os **Autômatos Finitos** (AFDs) demonstraram possuir o poder computacional mais restrito. Por não possuírem memória auxiliar, sua aplicação limita-se a reconhecer padrões sequenciais e repetições (como L_1). Eles não conseguem, por exemplo, contar ou balancear símbolos. A tentativa de processar $a^n b^n$ com um AFD exigiria um número infinito de estados, o que viola a sua definição.

Os **Autômatos de Pilha** (APs) resolvem o problema da contagem ao adicionar a estrutura de pilha (memória finita em profundidade). Com isso, tornam-se capazes de reconhecer pares balanceados, como casar os 'a's com os 'b's (L_2). Contudo, a limitação de acesso estrito ao topo da pilha impede que o AP reconheça padrões complexos de cruzamento tridimensional, tornando-o incapaz de lidar com linguagens como $a^n b^n c^n$.

As **Máquinas de Turing** (MT) demonstraram o ápice do poder computacional. Com a introdução de uma fita de leitura/escrita infinita e um cabeçote móvel para a esquerda e direita, a MT atinge o conceito de completude de Turing. Esta máquina não apenas consegue memorizar e contar, mas também remarcar dados lidos anteriormente (essencial para reconhecer L_4) e realizar cálculos numéricos absolutos, como comprovado pela execução da máquina de adição unária.

6 Conclusão

A execução deste projeto proporcionou uma solidificação inestimável dos conceitos abordados na disciplina. Traduzir o jargão matemático rigoroso (como 5-tuplas e relações de transição) para código limpo e funcional em C# quebrou a abstração teórica, provando que matrizes, dicionários e pilhas do cotidiano de desenvolvimento de software são, no fundo, modelos matemáticos.

Compreender o porquê de um JSON Parser ou Compilador precisar de uma árvore estruturada em vez de um simples RegEx (pois RegEx não tem memória de pilha) muda a forma de arquitetar softwares no mercado de trabalho. A teoria da computação deixa de ser uma disciplina isolada e consolida-se como a base irrefutável de qualquer processo de engenharia de software moderno.

Referências

- [1] MENEZES, P. B. *Linguagens formais e autômatos*. 6. ed. Porto Alegre: Bookman, 2010.
- [2] HOPCROFT, J. E.; MOTWANI, R.; ULLMAN, J. D. *Introdução à teoria de autômatos, linguagens e computação*. Rio de Janeiro: Elsevier, 2002.
- [3] SIPSER, M. *Introdução à teoria da computação*. São Paulo: Cengage Learning, 2007.